# Building LLM - Relazione generale del progetto

## Introduzione

Building LLM è un progetto didattico che parte dal modello linguistico più semplice e comprensibile e procede, una versione alla volta, verso uno small LLM decoder-only moderno.

Ogni versione introduce un solo concetto mirato, mantenendo i meccanismi visibili, eseguibili e adatti allo studio.

```
una versione -> un concetto -> un risultato verificabile
```

---

## Obiettivo del progetto

L'obiettivo non è competere con modelli industriali. Lo scopo è comprendere come i modelli linguistici vengono rappresentati, addestrati, valutati, sottoposti a fine-tuning e utilizzati, passando dai conteggi statistici a un Transformer decoder-only.

Il progetto segue questi principi:

- l'inglese è la lingua iniziale del modello;
- i primi meccanismi sono implementati dalle basi;
- la complessità aumenta gradualmente e resta spiegabile;
- i notebook sono eseguibili dall'alto verso il basso;
- la riproducibilità conta più della scala del modello;
- le fasi avanzate restano adatte a hardware Kaggle accessibile.

---

## Vincoli del progetto

- la Versione 1 usa esclusivamente Python standard;
- NumPy viene introdotto prima di un framework di machine learning;
- TensorFlow appare solo dopo aver compreso una piccola rete neurale;
- i dataset pubblici richiedono origine, versione e licenza documentate;
- ogni versione mantiene controlli adeguati al proprio ambito ed evita infrastruttura production non necessaria.

## Che cosa significa LLM in questo percorso

Le prime versioni sono modelli linguistici, ma non sono ancora large language model. Nel progetto, LLM rappresenta la destinazione didattica: un Transformer decoder-only con rappresentazioni subword apprese, training su dati pubblici, selezione tramite checkpoint, generazione controllata, instruction tuning e multi-head attention.

Il modello risultante rimane volutamente piccolo. Il nome descrive l'architettura e il percorso di apprendimento, non una scala industriale di parametri.

---

## Il ciclo stabile del modello linguistico

Nel progetto rimane stabile il seguente ciclo concettuale:

```
testo -> rappresentazione -> contesto
-> probabilità del token successivo
-> decoding -> testo generato
```

La Versione 1 ottiene le probabilità direttamente dai conteggi osservati. Le versioni successive introducono parametri addestrabili, ottimizzazione, contesti più ricchi, stato ricorrente, attention, tokenizzazione subword, training con checkpoint, valutazione held-out, supervised instruction tuning e infine più head di attention causale.

---

## Perché il progetto evolve in modo incrementale

I notebook evitano volutamente di sostituire molti meccanismi nello stesso passaggio. Una nuova versione preserva l'esperimento precedente quando possibile, così l'effetto di una sola idea può essere osservato direttamente. Questo rende il progetto utile sia come percorso didattico sia come portfolio tecnico: ogni notebook ha uno scopo chiaro, un ambito limitato e uno stato finale verificabile.

# La vera base della Versione 1

La Versione 1 è un modello linguistico statistico a caratteri scritto con Python standard. Legge un corpus inglese incorporato, crea coppie di caratteri adiacenti, conta le transizioni, normalizza i conteggi in probabilità e campiona un carattere alla volta.

I controlli implementati verificano il numero previsto di esempi, la normalizzazione delle probabilità, la riproducibilità della generazione con lo stesso seed e la lunghezza richiesta dell'output generato.

La Versione 1 non contiene componenti neurali addestrabili. Stabilisce il ciclo del language modeling prima di introdurre l'ottimizzazione.

---

# Roadmap - Fondamenti statistici e neurali

## Versione 1 - Modello statistico a caratteri

Conta le transizioni tra caratteri adiacenti, converte i conteggi in probabilità e genera testo con sampling riproducibile usando Python standard.

## Versione 2 - Piccola rete neurale con NumPy

Sostituisce la tabella dei conteggi con un piccolo modello neurale e rende visibile l'apprendimento dei parametri tramite operazioni NumPy.

## Versione 3 - Fondamenti del training

Introduce batch, suddivisione dei dati, un obiettivo di training, ottimizzazione e riproducibilità sistematica.

## Versione 4 - Introduzione a TensorFlow

Ricostruisce il piccolo modello con TensorFlow e differenziazione automatica dopo averne compreso i calcoli fondamentali.

## Versione 5 - Embedding e finestre di contesto

Apprende rappresentazioni vettoriali e usa più di un carattere precedente come contesto.

# Roadmap - Sequence modeling e attention

## Versione 6 - Modello linguistico ricorrente

Introduce una RNN semplice e uno stato nascosto che trasporta informazione nella sequenza.

## Versione 7 - Modello linguistico GRU

Aggiunge gated recurrent units per controllare come l'informazione viene mantenuta e aggiornata.

## Versione 8 - Attention

Costruisce la scaled dot-product attention da componenti comprensibili e osserva come le posizioni scambiano informazione.

## Versione 9 - Blocco Transformer

Combina self-attention causale, livelli feed-forward, connessioni residue e normalizzazione.

## Versione 10 - Transformer decoder-only

Impila blocchi Transformer e realizza generazione autoregressiva con mascheramento causale.

---

# Continuità progettuale

Il passaggio dalla ricorrenza all'attention non elimina le lezioni precedenti. Contesto, obiettivo next-token, distribuzioni di probabilità, training basato sui gradienti e riproducibilità restano visibili mentre il meccanismo che combina informazione tra posizioni diventa progressivamente più espressivo.

# Roadmap - Dati pubblici, training, valutazione e adattamento

## Versione 11 - Tokenizzazione subword e dataset pubblico

Passa dai caratteri a un vocabolario BPE da 2.048 token addestrato solo sul testo di training WikiText-103 selezionato, preservando i confini ufficiali di train, validation e test.

## Versione 12 - Pipeline di training e checkpoint

Rende esplicito lo stato di training. LAST risponde a dove riprendere il training; BEST risponde a quale modello viene selezionato dalla validation. Una interruzione simulata pulita ripristina LAST e continua fino al budget di 20 epoche, prima di ricostruire un nuovo modello da BEST per la valutazione finale.

## Versione 13 - Valutazione e generazione controllata

Mantiene invariato il training della Versione 12 e aggiunge cross-entropy held-out, perplexity, accuracy top-1/top-5, temperatura, top-k, top-p e sampling autoregressivo riproducibile.

## Versione 14 - Instruction tuning

Aggiunge una piccola fase di supervised fine-tuning con Databricks Dolly 15K. La formattazione esplicita instruction/context/response e una response mask limitano la loss supervisionata ai target della risposta, mentre la validation seleziona il miglior checkpoint instruction.

### Versione 15 - Transformer multi-head

Preserva l'intera pipeline della Versione 14 e divide lo spazio di attention esistente da 16 dimensioni in due head causali indipendenti da 8 dimensioni. Le shape delle proiezioni e il totale di 34.496 parametri restano invariati, isolando la multi-head attention come concetto architetturale più recente.

## Strategia per i dataset

La Versione 1 incorpora direttamente nel notebook un piccolo corpus didattico. La Versione 11 sposta il progetto su WikiText-103 Raw v1, usando subset controllati e deterministici pur preservando i confini ufficiali degli split.

Il tokenizer BPE viene addestrato soltanto sul testo WikiText di training selezionato. Validation e test non influenzano la costruzione del vocabolario. La Versione 14 aggiunge Databricks Dolly 15K come dataset separato di istruzioni; il tokenizer BPE esistente viene riutilizzato e non riaddestrato sui dati instruction.

Per i dataset reali il progetto documenta sorgente, identità del dataset, lingua, scelte di preprocessing e ruolo dei dati di train, validation e test.

---

## Strategia Kaggle

- notebook eseguibili dall'alto verso il basso;
- seed e valori di configurazione principali espliciti;
- dataset esterni collegati sotto /kaggle/input e input mancanti gestiti con stop esplicito, senza fallback silenziosi;
- directory dei checkpoint sotto /kaggle/working e ricreate per run didattici puliti;
- dimensioni di modello e dataset conservative per hardware Kaggle accessibile.

---

## Qualità e documentazione

Ogni versione termina con controlli adeguati al proprio ambito e una sezione Notes che descrive cosa è cambiato, cosa è rimasto invariato e quali limitazioni restano. Le versioni avanzate separano esplicitamente training, selezione tramite validation e reporting held-out sul test. Il progetto documenta le semplificazioni deliberate invece di presentarle come funzionalità production.

## Risultato attuale

Lo stato attuale del progetto è uno small LLM inglese decoder-only addestrato e sottoposto a fine-tuning su hardware Kaggle accessibile. La pipeline attuale combina:

- un tokenizer BPE appreso dal testo pubblico WikiText di training;
- contesti causali fissi da 16 token e un Transformer decoder-only a due blocchi;
- semantica esplicita dei checkpoint LAST/BEST e ripresa del training dentro un run pulito;
- valutazione held-out con cross-entropy, perplexity e token accuracy;
- generazione controllata con temperatura, top-k, top-p, greedy decoding e sampling riproducibile;
- supervised instruction tuning con loss mascherata focalizzata sulla risposta;
- self-attention causale a due head mantenendo l'architettura compatta da 34.496 parametri.

Il progetto resta intenzionalmente piccolo e trasparente. Non dichiara scala commerciale, infrastruttura di serving production, RLHF, agenti o ottimizzazione su larga scala. L'obiettivo è una progressione coerente ed eseguibile in cui ogni versione insegna un concetto identificabile e si chiude con un risultato verificabile.

---

## Sintesi del ciclo attuale

Le Versioni 1-15 formano un arco didattico completo: modellazione statistica -> apprendimento neurale -> sequence modeling -> attention -> Transformer decoder-only -> dati pubblici subword -> training con checkpoint -> valutazione e generazione controllata -> instruction tuning -> self-attention causale multi-head.

Questa struttura attuale è coerente con i notebook implementati e offre una narrazione portfolio pulita, senza aggiungere complessità non dimostrata dal codice.

Il progetto resta aperto a versioni future, che potranno estendere architettura, training, ottimizzazione e alignment mantenendo la stessa progressione incrementale.